

Guião 04 – Docker: Primeiros Contentores

Infraestruturas de Computação na Nuvem, Aula 04

1 Objetivos

- Observar diretamente as primitivas de isolamento do kernel (namespaces e cgroups) usadas por um contentor.
- Correr contentores a partir de imagens públicas do Docker Hub.
- Escrever um Dockerfile simples e construir uma imagem própria.
- Usar volumes para persistir dados fora do ciclo de vida do contentor.
- Criar uma rede Docker do tipo bridge e ligar dois contentores através dela.
- Comparar um contentor de aplicação (Docker) com um contentor de sistema (LXC do Proxmox).

2 Pré-requisitos e Material

- A VM **icn-lab** no Proxmox, criada no Guião 01, com o Docker já instalado e acessível por SSH.
- Acesso à Internet a partir da VM, para descarregar imagens do Docker Hub.
- Editor de texto (**nano**, **vim**) dentro da VM, para escrever o Dockerfile e o código da aplicação.
- Acesso à interface web do Proxmox, para a comparação final com LXC.

3 Introdução

Nesta aula prática vamos explorar Docker enquanto ferramenta de gestão de contentores. Todo o trabalho decorre **dentro da VM icn-lab** preparada no Guião 01 – ou seja, vamos correr contentores dentro de uma máquina virtual, que por sua vez corre sobre o hypervisor KVM do servidor Proxmox. Esta sobreposição de camadas é intencional e comum em infraestruturas reais de cloud.

Começamos por observar concretamente os namespaces e cgroups estudados na teoria, passamos pelo ciclo **pull / run / stop**, escrevemos o nosso próprio Dockerfile, e terminamos com volumes, redes personalizadas e uma comparação direta com o contentor LXC visto na Aula 03.

4 Passo a Passo

1. **Verificar o Docker na VM.** Estabeleça a ligação SSH à sua VM e confirme que o Docker instalado no Guião 01 está operacional:

```
$ ssh utilizador@IP_da_VM
$ docker --version
$ docker run hello-world
```

O último comando descarrega uma pequena imagem de teste e confirma que o daemon está a funcionar corretamente.

2. **Observar os namespaces de um contentor.** Arranque um contentor de longa duração e compare a sua visão do sistema com a da VM que o hospeda:

```
$ docker run -d --name observar alpine sleep 3600
```

Compare a árvore de processos vista de dentro e de fora do contentor:

```
# Dentro do contentor (namespace PID proprio):
$ docker exec observar ps aux

# Fora, na VM (o mesmo processo, outro PID):
$ ps aux | grep sleep
```

Registe o PID que o processo `sleep` tem *dentro* do contentor e o PID que tem *na VM*. Esta diferença é o namespace PID em ação.

3. **Listar os namespaces atribuídos ao contentor.** Obtenha o PID do processo no host e inspecione os seus namespaces:

```
$ PID=$(docker inspect -f '{{.State.Pid}}' observar)
$ sudo ls -l /proc/$PID/ns
```

Identifique na listagem os namespaces `pid`, `net`, `mnt`, `uts` e `ipc` estudados na aula teórica. Compare com os do processo `init` da própria VM:

```
$ sudo ls -l /proc/1/ns
```

Os identificadores numéricos que diferem correspondem aos recursos efetivamente isolados.

4. **Observar o isolamento de rede e de hostname.**

```
$ docker exec observar hostname
$ docker exec observar ip addr show
$ hostname
$ ip addr show
```

O contentor tem hostname e interfaces de rede próprios – namespaces UTS e NET.

5. **Aplicar e verificar limites com cgroups.** Corra um contentor com memória limitada e confirme o limite efetivamente aplicado pelo kernel:

```
$ docker run -d --name limitado --memory=256m --cpus=0.5 alpine sleep 3600
$ docker stats --no-stream
```

Confirme o limite diretamente no sistema de ficheiros de cgroups da VM:

```
$ PID2=$(docker inspect -f '{{.State.Pid}}' limitado)
$ cat /proc/$PID2/cgroup
```

Consoante a versão de cgroups em uso, o valor do limite de memória pode ser consultado em `/sys/fs/cgroup/...` (v2, hierarquia unificada) ou em `/sys/fs/cgroup/memory/...` (v1). Verifique qual a versão ativa na sua VM:

```
$ stat -fc %T /sys/fs/cgroup
```

O resultado `cgroup2fs` indica cgroups v2; `tmpfs` indica v1.

6. **Limpar os contentores de observação.**

```
$ docker rm -f observar limitado
```

7. **Correr um contentor nginx.** Descarregue e execute a imagem oficial do nginx, publicando a porta 80 do contentor na porta 8080 do host:

```
$ docker run -d --name web-teste -p 8080:80 nginx
```

Confirme no navegador (<http://localhost:8080>) que o servidor responde. Liste os contentores em execução e pare o contentor no final:

```
$ docker ps
$ docker stop web-teste
$ docker rm web-teste
```

8. **Explorar um contentor alpine interativo.** Alpine é uma imagem base minimalista, muito usada como ponto de partida para outras imagens:

```
$ docker run -it --rm alpine sh
/ # cat /etc/os-release
/ # exit
```

A opção `-rm` remove automaticamente o contentor assim que este termina.

9. **Escrever uma pequena aplicação.** Crie uma pasta `app-teste/` com um ficheiro `app.py` com uma aplicação Flask muito simples:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Ola a partir do meu contentor!\n"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

E um ficheiro `requirements.txt`:

```
flask
```

10. **Escrever o Dockerfile.** No mesmo diretório, crie um ficheiro `Dockerfile`:

```
FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

11. **Construir a imagem.** A partir do diretório `app-teste/`:

```
$ docker build -t minha-app:1.0 .
$ docker images
```

12. **Correr a imagem localmente.**

```
$ docker run -d --name minha-app -p 5000:5000 minha-app:1.0
$ curl http://localhost:5000
```

Confirme que a resposta corresponde ao texto definido em `app.py`.

13. **Usar um volume para persistência.** Crie um volume nomeado e monte-o num contentor, para que os dados sobrevivam à remoção do contentor:

```
$ docker volume create dados-teste
$ docker run -it --rm -v dados-teste:/dados alpine sh
/ # echo "conteudo persistente" > /dados/ficheiro.txt
/ # exit
$ docker run -it --rm -v dados-teste:/dados alpine cat /dados/ficheiro.txt
```

Note que o ficheiro criado no primeiro contentor está disponível no segundo, mesmo o primeiro já tendo sido removido.

14. **Criar uma rede bridge personalizada.**

```
$ docker network create rede-teste
$ docker network ls
```

15. **Ligar dois contentores através da rede personalizada.** Corra dois contentores nessa rede e confirme que conseguem comunicar pelo nome:

```
$ docker run -d --name servidor --network rede-teste nginx
$ docker run -it --rm --network rede-teste alpine sh
/ # wget -O- http://servidor
/ # exit
```

Ao contrário da rede bridge por omissão, uma rede personalizada permite resolução de nomes entre contentores pelo nome do contentor.

16. **Comparar com um contentor de sistema (LXC).** Na aula anterior criámos um contentor LXC no Proxmox. Compare os dois modelos:

```
# No contentor Docker (aplicacao): um unico processo
$ docker run -d --name comparar nginx
$ docker exec comparar ps aux
```

Se ainda tiver (ou recriar) um contentor LXC no Proxmox, observe a diferença:

```
# No node Proxmox, dentro do contentor LXC (sistema):
$ pct enter <ctid>
root@icn-ct:~# ps aux
root@icn-ct:~# systemctl list-units --type=service | head
```

Registe quantos processos correm em cada caso. O contentor Docker tem tipicamente um processo (mais os seus filhos diretos); o LXC tem um sistema de init completo com vários serviços.

17. **Limpeza final.**

```
$ docker rm -f servidor minha-app comparar
$ docker network rm rede-teste
$ docker volume rm dados-teste
```

5 Entregável / Questões de Verificação

- a) Registe o PID do processo `sleep` visto *dentro* do contentor e *na* VM. Explique a diferença com base no namespace PID.
- b) Compare a saída de `ls -l /proc/<pid>/ns` para o contentor e para o processo 1 da VM. Que namespaces diferem, e o que significa cada um deles estar isolado?
- c) Qual a versão de cgroups ativa na sua VM, e como a determinou? Que limite de memória foi efetivamente aplicado ao contentor `limitado`?
- d) Qual a diferença entre a imagem `minha-app:1.0` e o contentor criado a partir dela?
- e) Porque é que o ficheiro criado no volume `dados-teste` sobreviveu à remoção do primeiro contentor?
- f) Explique, por palavras suas, porque é que dois contentores na rede bridge por omissão do Docker não se conseguem descobrir mutuamente pelo nome, ao contrário do que acontece numa rede personalizada.
- g) Quantos processos observou no contentor Docker e quantos no contentor LXC? Relacione com a distinção entre contentor de aplicação e contentor de sistema.
- h) Nesta aula, um contentor Docker corre dentro de uma VM KVM, que corre sobre o servidor Proxmox. Enumere as camadas de virtualização envolvidas e indique, para cada uma, se partilha ou não o kernel com a camada inferior.
- i) Envie o `Dockerfile` utilizado, juntamente com uma captura de ecrã do comando `curl` sobre `localhost:5000` com a resposta da aplicação.